

REVISÃO DA LITERATURA

Trabalho Prático 2 - Pipeline RISC-V

Practical Exercise 2 - RISC-V Pipeline

Davi Cândido de Almeida   [Pontifícia Universidade Católica de Minas Gerais | davicandidopucminas@gmail.com]

 Pontifícia Universidade Católica de Minas Gerais, Rua Dom José Gaspar, nº500 - Coração Eucarístico, Belo Horizonte - MG, 30535-901, Brasil.

Resumo. Este trabalho, teve como foco completar partes importantes de um processador RISC-V em pipeline. Implementando um desvio (BEQ) para que o processador redirecione o PC corretamente quando a condição é verdadeira e descarte instruções que ficaram no caminho errado. Também foi implementado forwarding, que reaproveita resultados sem precisar esperar a conclusão da instrução pela CPU, e a detecção de dependências que insere uma pausa (stall) só quando realmente necessário. Depois disso, testes foram executados sobre o programa a partir de resultados esperados, mostrando que a execução ficou correta e o comportamento do pipeline ficou de acordo com o objetivo do trabalho.

Abstract. This work focused on completing important parts of a pipelined RISC-V processor. It implemented a branch (BEQ) so that the processor correctly redirects the PC when the condition is true and discards instructions that got lost in the wrong path. Forwarding was also implemented, which reuses results without waiting for the CPU to complete the instruction, and dependency detection was implemented, inserting a stall only when truly necessary. Afterwards, tests were run on the program based on expected results, showing that the execution was correct and the pipeline behavior was in accordance with the work's objective.

Palavras-chave: Pipeline; RISC-V; Hazard de dados; RAW; Forwarding; Bypass; Stall; Load-use hazard; Hazard de controle; Branch; Flush; CPI; Verilog; Arquitetura de computadores; Simulação.

Keywords: Pipeline; RISC-V; Data hazard; RAW; Forwarding; Bypass; Stall; Load-use hazard; Control hazard; Branch; Flush; CPI; Verilog; Computer architecture; Simulation.

1 Introdução

Este trabalho teve como objetivo implementar e analisar mecanismos fundamentais de execução em pipeline em um processador RISC-V didático de 5 estágios (IF, ID, EX, MEM, WB). O foco foi o tratamento de hazards de dados e de controle, por meio de forwarding, stalls e flush, além da avaliação de impacto no desempenho via métricas de simulação (ciclos, instruções, stalls, bypasses, branches, flushes e CPI).

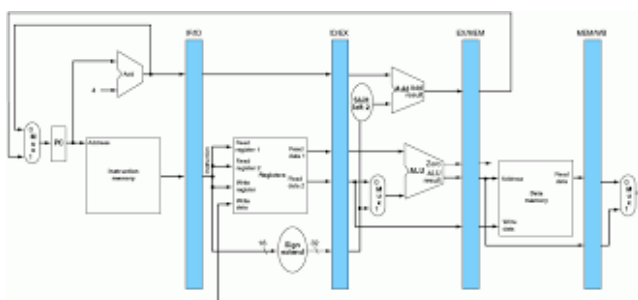


Figura 1. Visão geral do pipeline de 5 estágios utilizado no trabalho.

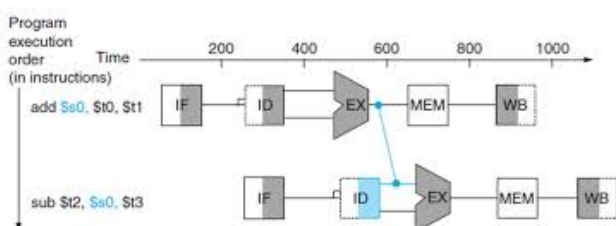


Figura 2. Visão didática do pipeline de 5 estágios utilizado no trabalho.

2 Desenvolvimento

O projeto utiliza um processador RISC-V simplificado, não sintetizável, para fins didáticos. A execução ocorre em pipeline com registradores entre estágios e módulos auxiliares de controle:

- BranchUnit
- ForwardingUnit
- HazardDetectionUnit
- PipelineStats

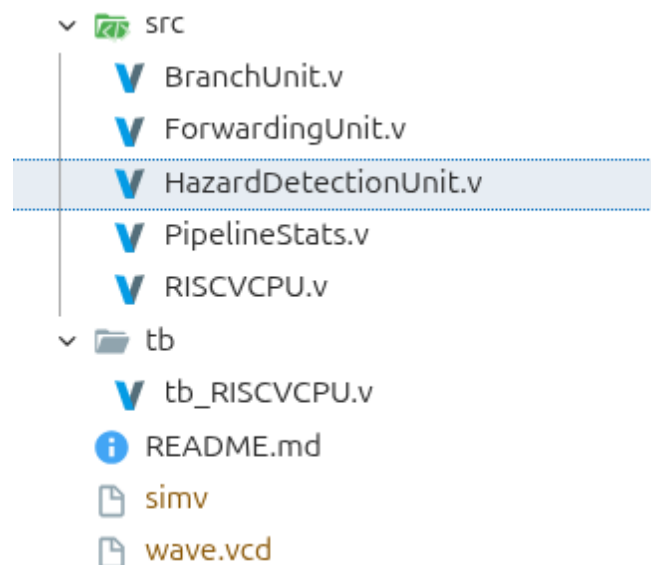


Figura 3. Estrutura de pastas / arquivos do repositório

2.1 Parte 1 - Branch e controle de fluxo

Foi implementada a lógica de branch condicional BEQ no módulo BranchUnit, com:

- identificação do opcode de branch;
- identificação do opcode de branch;
- cálculo de endereço alvo ($branch_target = pc_ex + branch_imm$);
- sinalização de branch tomado ($branch_taken$).
- atualiza o PC para o alvo, quando o branch é tomado.

```

1  always @(*) begin
2      branch_taken = 1'b0;
3      branch_target = pc_ex + 32'd4;
4
5
6      if (opcode == BEQ) begin
7          if (rs1_value == rs2_value)
8              begin
9                  branch_taken = 1'b1;
10                 branch_target = pc_ex +
11                     branch_imm;
12             end
13         end
14     end
15 end

```

2.2 Parte 2 - Execução do programa de teste

A etapa 2 foi analisada no contexto incremental: sem controle de hazards, o programa apresenta resultados incorretos devido a dependências RAW. Isso evidencia a necessidade de reordenação/NOPs ou, preferencialmente na etapa seguinte, implementação explícita de forwarding + stall.

2.3 Parte 3 - Hazard Detection e Forwarding

Foram implementadas duas lógicas principais:

- Forwarding (ForwardingUnit)
 - seleção de operandos da ALU com prioridade para EX/MEM e depois MEM/WB;
 - distinção entre forwarding de resultado de ALU e de load em WB;
 - proteção para não encaminhar x0;
 - bloqueio de forwarding de EX/MEM quando a instrução é LW (pois nesse ponto ainda há endereço, não dado carregado).
- Detecção de hazard (HazardDetectionUnit)
 - detecção de load-use: se a instrução à frente é LW e o registrador destino coincide com uma fonte da instrução seguinte, insere-se stall = 1 por 1 ciclo.

```

1  always @(*) begin
2      branch_taken = 1'b0;
3      branch_target = pc_ex + 32'd4;
4
5
6      if (opcode == BEQ) begin

```

```

7          if (rs1_value == rs2_value)
8              begin
9                  branch_taken = 1'b1;
10                 branch_target = pc_ex +
11                     branch_imm;
12             end
13         end
14     end
15 end

```

```

1  always @(*) begin
2      forwardA = NO_FORWARD;
3      forwardB = NO_FORWARD;
4
5      if ((exmem_rd != 5'd0) && (exmem_rd ==
6          idex_rs1) && (exmem_op != LW)) begin
7          forwardA = FROM_MEM;
8      end
9      else if ((memwb_rd != 5'd0) && (memwb_rd
10         == idex_rs1)) begin
11         if (memwb_op == LW)
12             forwardA = FROM_WB_LD;
13         else
14             forwardA = FROM_WB_ALU;
15     end
16     if ((exmem_rd != 5'd0) && (exmem_rd ==
17         idex_rs2) && (exmem_op != LW)) begin
18         forwardB = FROM_MEM;
19     end
20     else if ((memwb_rd != 5'd0) && (memwb_rd
21         == idex_rs2)) begin
22         if (memwb_op == LW)
23             forwardB = FROM_WB_LD;
24         else
25             forwardB = FROM_WB_ALU;
26     end
27 end
28 end

```

```

1  always @(*) begin
2      stall = 1'b0;
3
4      if ((exmem_op == LW) && (exmem_rd !=
5          5'd0) &&
6          ((exmem_rd == idex_rs1) || (
7              exmem_rd == idex_rs2)))
8          begin
9              stall = 1'b1;
10          end
11      end
12  end

```

3 Resultados e validação

A simulação final foi executada com:

- iverilog -o simv src/*.v tb/tb_RISCVCPU.v
- vvp simv

Resultado funcional:

- PASS: all expected results match

Estado final relevante:

- x1 = 10
- x2 = 15
- x3 = 16
- x4 = 26
- x7 = 27
- mem[1] = 16

Métricas:

- Ciclos: 15
- Instruções: 7
- Stalls: 1
- Bypasses: 5
- Branches tomados: 1
- Flushes: 1
- CPI: 2.14

VCD info: dumpfile wave.vcd opened for output.

```
=====
Program      : full_dependencies
Cycles       : 15
Instructions  : 7
Stalls       : 1
Bypasses     : 5
Branches     : 1
Flushes      : 1
CPI          : 2.14
=====
```

```
=====
CHECK EXPECTED RESULTS
=====
PASS: all expected results match.
=====
```

```
=====
STATE (non-zero values) for DEBUG
=====
```

```
-- REGISTERS --
x1 = 10 (0x0000000a)
x2 = 15 (0x0000000f)
x3 = 16 (0x00000010)
x4 = 26 (0x0000001a)
x7 = 27 (0x0000001b)

-- DATA MEMORY --
mem[0] = 10 (0x0000000a)
mem[1] = 16 (0x00000010)
=====
```

Figura 4. Resultados obtidos via terminal

4 Análise

4.1 Por que stalls ocorrem

Stalls ocorrem quando o dado necessário ainda não está disponível no momento em que a instrução dependente precisa executar. No teste, isso acontece no caso clássico load-use: a instrução seguinte usa um valor carregado por lw antes de ele estar pronto para uso direto em EX.

4.2 Quando forwarding resolve dependências

Forwarding resolve dependências quando o valor já foi produzido em estágios mais avançados (EX/MEM ou MEM/WB), evitando esperar a escrita no banco de registradores. No experimento, 5 dependências foram resolvidas por bypass

4.3 Impacto de branch no desempenho

Branches impactam desempenho por risco de controle: instruções buscadas especulativamente no caminho sequencial podem precisar ser descartadas (flush). No teste, houve 1 branch tomado e 1 flush, com penalidade observável em ciclos extras.

4.4 Relação com CPI

O CPI observado (2.14) reflete o efeito combinado de:

- enchimento/esvaziamento do pipeline,
- 1 stall por load-use,
- 1 flush por branch tomado. Mesmo com forwarding reduzindo penalidades de dados, hazards residuais ainda aumentam o CPI acima de 1.

5 Dificuldades e decisões de projeto

- Principal dificuldade: entender a diferença temporal entre dado de ALU e dado de lw.
- Decisão crítica: impedir forwarding de EX/MEM para lw, pois o valor correto do load só aparece após MEM.
- Estratégia de validação: usar o testbench fornecido e conferir tanto o PASS quanto registradores/memória e métricas.

6 Conclusão

O trabalho permitiu consolidar conceitos fundamentais de pipeline: execução sobreposta, hazards de dados e controle, forwarding, stalls e flush. A implementação final atingiu comportamento funcional correto no cenário de teste e demonstrou, por métricas, os custos e benefícios das técnicas de resolução de dependências.

7 Uso de Inteligência Artificial (IA)

7.1 Transparência

Nesta etapa do trabalho, utilizei ferramentas de IA como **apoio de estudo e orientação**, principalmente para organizar o raciocínio, esclarecer conceitos e revisar decisões de implementação. A validação final foi feita por **simulação** com o testbench do projeto e pela conferência dos resultados esperados (registradores, memória e métricas), garantindo que o comportamento estivesse correto de forma objetiva.

7.2 Como a IA foi utilizada

A IA foi usada para:

- **Explicar** pipeline, hazards e o papel de forwarding/stall/flush com linguagem mais acessível;
- **Orientar** a leitura do código (módulos, sinais e fluxo entre estágios);

- **Sugerir caminhos** de implementação e checagens (o que testar e o que observar quando algo falha);
- **Revisar clareza** de comentários no código para facilitar a leitura e a organização da documentação do trabalho.

7.3 Prompts (modelo SDD) e contribuição para o entendimento

Prompt 0 – Entendimento geral do trabalho + aula guiada

Problema: Me considere como se fosse leigo em Verilog e em conceitos avançados de arquitetura de computadores; preciso entender o Trabalho Prático 2 (pipeline RISC-V) e o que cada arquivo faz, me guie no que preciso estudar antes de iniciar o trabalho:

Requisitos Funcionais Explicar o escopo do trabalho (Partes 1–3) com base no enunciado. Explicar minuciosamente o que preciso saber antes de tentar entender os código já existente (BranchUnit, ForwardingUnit, integração no RISCVCPU, testbench).

Requisitos não funcionais : Linguagem didática, passo a passo, adequada a quem nunca usou Verilog.

Restrições: Basear-se no projeto fornecido e no PDF do trabalho; não inventar requisitos fora do escopo.

Como usei: Para montar o “mapa mental” do pipeline, entender IF/ID/EX/MEM/WB e localizar onde cada parte do TP se encaixa.

Prompt 1 – Entendimento do trabalho e do código-base

Problema: Preciso entender o escopo do Trabalho Prático 2, mas precisamente os códigos disponíveis e como o projeto organiza o pipeline RISC-V.

Requisitos: Explicar as partes do enunciado e o papel dos arquivos principais (BranchUnit, ForwardingUnit, HazardDetectionUnit, RISCVCPU, testbench).

Restrições: Manter explicação didática para quem está começando em Verilog.

Como ajudou: Complementou o “mapa” do fluxo IF/ID/EX/MEM/WB e onde cada mecanismo se encaixa nos códigos disponibilizados, me guiando no que deveria ser feito.

Prompt 2 – Orientação para BEQ (Parte 1)

Problema: Preciso completar BranchUnit.v para suportar BEQ com comparação em EX e cálculo do destino.

Requisitos: Condiicionar a tomada de branch ao opcode e à igualdade dos operandos; calcular o alvo com o imediato correto.

Restrições: Respeitar a interface do módulo e o modelo já existente no projeto.

Como ajudou: Organizou a lógica de decisão e conectou com o efeito de flush já previsto no caminho de dados.

Prompt 3 – Conceitos em linguagem simples

Problema: Quero entender forwarding, stall e branch sem excesso de jargão.

Requisitos: Explicar com analogias e exemplos curtos ligados ao programa de teste.

Como ajudou: Facilitou escrever a parte conceitual do relatório e justificar por que *load-use* exige pausa.

Prompt 4 – Visualização do timing (tabela ciclo a ciclo)

Problema: Quero visualizar onde entra stall e onde ocorre flush no programa do testbench.

Requisitos: Tabela simplificada por ciclo com IF/ID/EX/MEM/WB.

Como ajudou: Transformou timing abstrato em algo verificável e fácil de explicar.

Prompt 5 – Validação por simulação e correção orientada

Problema: Quero testar o sistema completo e entender falhas quando o resultado não bate.

Requisitos: Rodar simulação, interpretar métricas e ajustar apenas o que for necessário, com justificativa técnica.

Como ajudou: Direcionou o diagnóstico.

Prompt 6 – Comentários objetivos no código

Problema: Quero comentários curtos que expliquem decisões-chave sem poluir o arquivo.

Requisitos: Destacar o “porquê” das condições principais.

Como ajudou: Melhorou a legibilidade e a preparação do documento explicativo das soluções.

Prompt 7 – Organização da entrega (relatório/checklist)

Problema: Quero um checklist do que o PDF pede além do código (análises, IA, repositório), me guie na organização da documentação do projeto e o que deverá ser apresentado em cada sessão.

Requisitos: Lista objetiva alinhada ao enunciado do trabalho.

Como ajudou: Evitou esquecer itens formais da entrega e a me guiar sobre como organizar a documentação.

7.4 Evidência de validação (objetiva)

Ao final, a simulação do testbench indicou execução correta conforme os checks do projeto (PASS), com métricas coerentes com o cenário (por exemplo, presença de stall em *load-use*, uso de bypass quando aplicável, branch tomado com flush).

Disponibilidade de artefatos de pesquisa

Os códigos e outros materiais criados e/ou usados para a seguinte atividade estão disponibilizados no seguinte repositório: <https://github.com/DaviKandido/Pipeline-RISC-V.git>